

Project Name: Legal Ease: AI-Powered Legal Document Generator

Team Members:

1. Anitha M (Team Leader)
3. Nithyasri M

2. Swathi M
4. Pooja Shree P

Project Overview & Objective

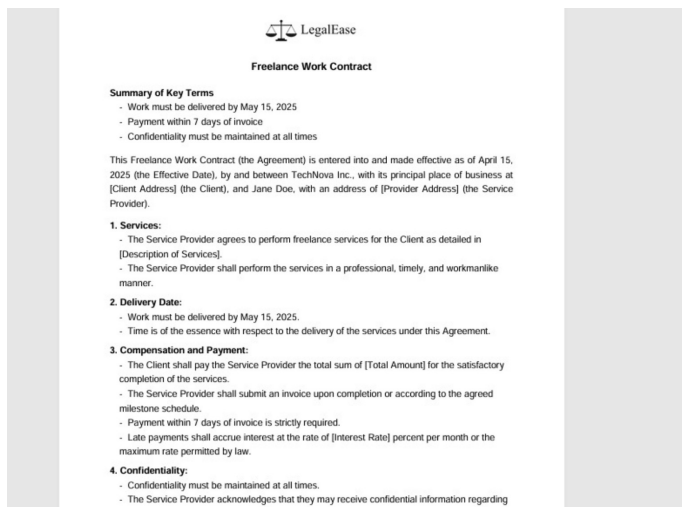
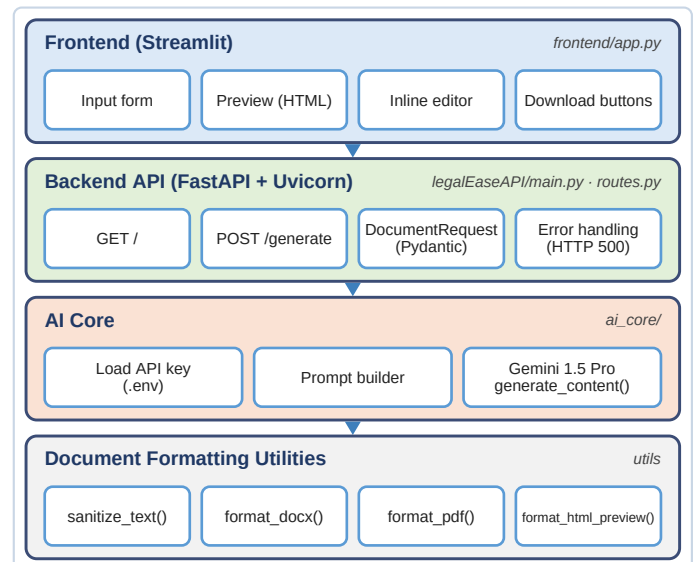
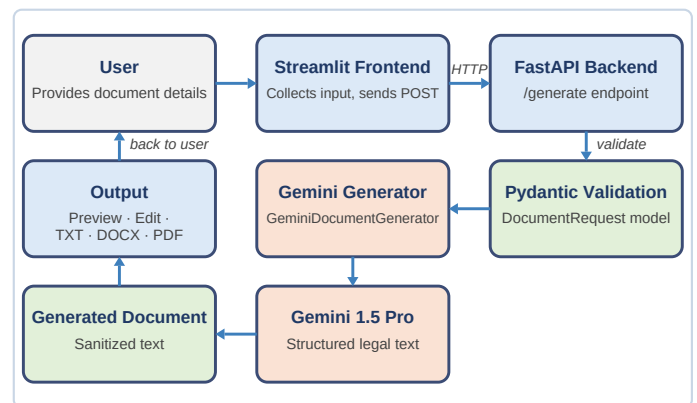
LegalEase is an AI-powered legal document generator developed using Python. It provides a simple web interface where users enter the document type, involved parties, terms and conditions, and effective date to generate a structured legal document.

Objective: Build a simple and useful AI-assisted system that helps users generate structured legal documents from the information they provide.

Main Features

- AI-based legal document generation
- User input for document type, parties, terms and effective date
- FastAPI REST endpoint and Gemini 1.5 Pro integration
- Generated document preview and inline editing
- Download as TXT, DOCX and PDF
- Document formatting and sanitization

Technologies: Python, Streamlit, FastAPI, Gemini 1.5 Pro, Pydantic, python-docx, fpdf2.



Generated legal-document output

Project Architecture & Architecture Description

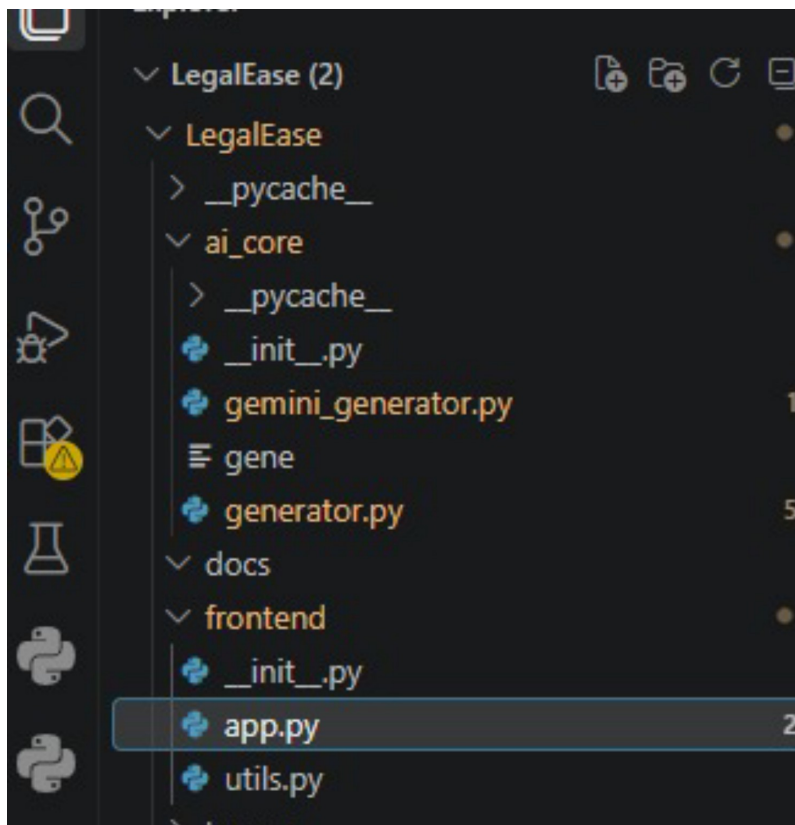
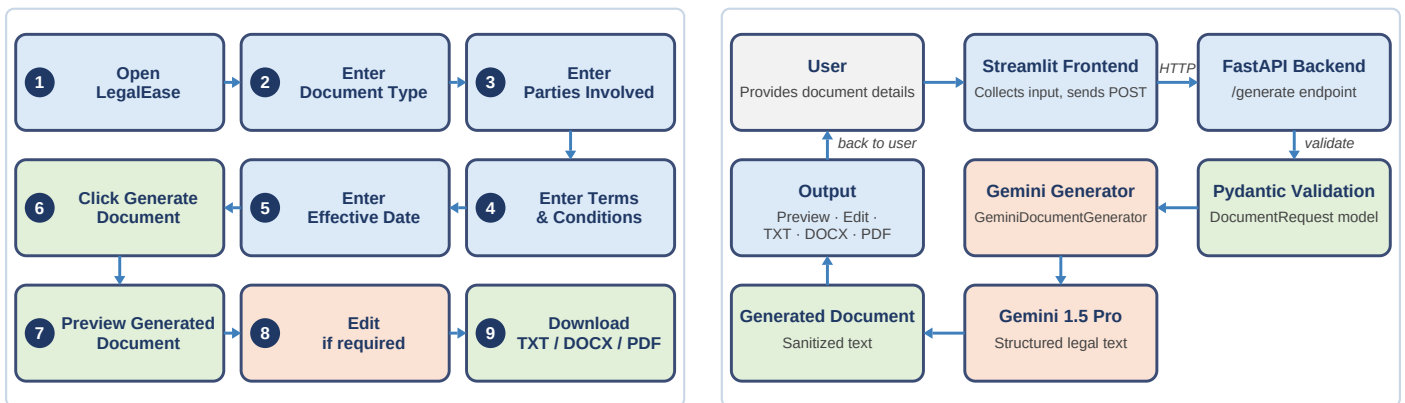
LegalEase follows a modular architecture: the user interface communicates with the backend API, the backend validates the request, and the AI core generates the legal document.

Architecture Flow

USER → STREAMLIT FRONTEND → HTTP POST /generate → FASTAPI BACKEND → GEMINI DOCUMENT GENERATOR → GEMINI 1.5 PRO → GENERATED LEGAL DOCUMENT → PREVIEW / EDIT / TXT / DOCX / PDF

Key Architecture Steps

- User enters the required legal document information.
- Streamlit collects the information and sends it to the backend.
- FastAPI receives the request through the /generate endpoint.
- Pydantic validates the document request data.
- Gemini Document Generator creates a structured prompt.
- Gemini 1.5 Pro generates the legal document text.
- The generated text is sanitized, previewed and edited before download.



Project folder structure

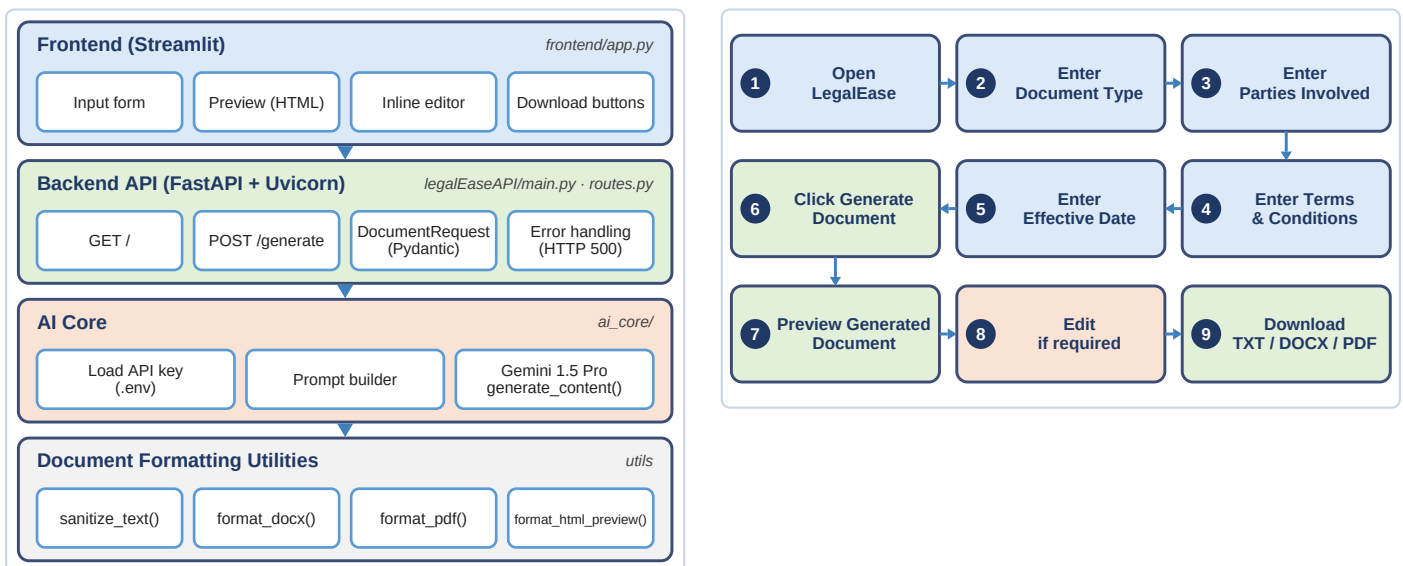
Technical Architecture & System Components

Technical Stack

Python — programming language; Streamlit — frontend framework; FastAPI — backend framework; Uvicorn — API server; Gemini 1.5 Pro — AI model; Pydantic — request validation; python-docx — DOCX creation; fpdf2 — PDF creation; Pillow — image processing; Requests — HTTP communication; python-dotenv — environment configuration.

System Components

- Frontend: collects document type, parties, terms and effective date, then displays the generated document.
- Backend API: provides /generate, receives validated JSON and passes values to the generator.
- AI Core: loads the API key, configures the Google Generative AI service, creates a structured prompt and calls Gemini 1.5 Pro.
- Document Formatting: sanitizes text and prepares DOCX, PDF and HTML preview output.



```
LegalEase > legalEaseAPI > _in
1 Generate code
  (Ctrl+I), or
  select a
  Language (Ctrl-
  M). Start typi
  to dismiss or
  don't show thi
  again.

LegalEase > requirements.txt
1 fastapi>=0.110
2 uvicorn[standard]>=0.29
3 streamlit>=1.40
4 python-docx>=1.1
5 fpdf2>=2.7.4
6 pillow>=10.0
7 requests>=2.31
8 google-gemini>=1.0
9 python-dotenv>=1.0
10 pydantic>=2.0

(venv) PS C:\Users\acer\Downloads\LegalEase (2)\LegalEase> streamlit run frontend/app.py
2026-09-24 21:23:43.653 Uvicorn server started on ::8503

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8503
Network URL: http://10.195.63.34:8503

Help agents write better Streamlit apps?
Install the official Streamlit skills by running streamlit skills in your terminal.
```

Required Python packages and runtime

Data Flow, Features & Responsibilities

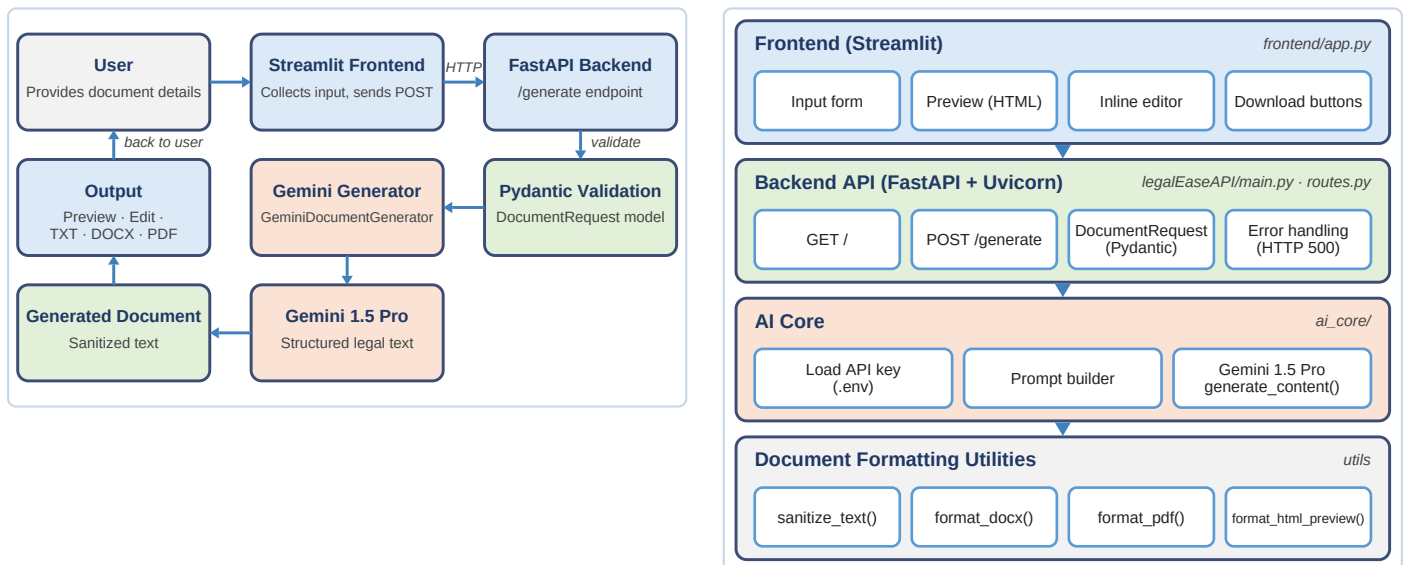
Data Flow: User Input → Streamlit Frontend → HTTP POST Request → FastAPI Backend → Pydantic Validation → Gemini 1.5 Pro → Generated Text → Sanitization → Preview / Edit → File Downloads.

Important Features

- Document type, parties, terms and effective date inputs
- REST API communication
- Gemini 1.5 Pro integration
- Preview and editing
- TXT, DOCX and PDF download
- Error and validation handling

Roles

- User: enters details and generates, edits, previews or downloads the document.
- Streamlit Frontend: collects input, calls the API and provides downloads.
- FastAPI Backend: validates requests and processes /generate.
- Gemini Generator: builds the AI prompt and generates legal text.
- Utility Module: sanitizes and formats TXT/DOCX/PDF/HTML outputs.



```
1 Generate code (Ctrl+I), or select a Language (Ctrl+M). Start typing to dismiss or don't show this again.
27 class GeminiDocumentGenerator:
28     def generate_document(self, document_type, parties, terms, dates) -> str:
29         message = response.json()["error"]["message"]
30         except Exception:
31             message = response.text[:300]
32             raise GeminiAPIError(f"{response.status_code}: {message}")
33
34         data = response.json()
35         candidates = data.get("candidates") or []
36         if not candidates:
37             raise RuntimeError(f"The model returned no answer: {data.get('promptFeedback')}")
38
39         parts = candidates[0].get("content", {}).get("parts", [])
40         text = "".join(p.get("text", "") for p in parts).strip()
41         if not text:
42             reason = candidates[0].get("finishReason", "unknown")
43             raise RuntimeError(f"The model returned an empty response (reason: {reason})")
44         return text
```

The screenshot also shows the Streamlit application running in the terminal, displaying the local URL: `http://localhost:8503` and the network URL: `http://10.195.63.34:8503`.

Gemini document-generator implementation

User Flow, Inputs & Project Structure

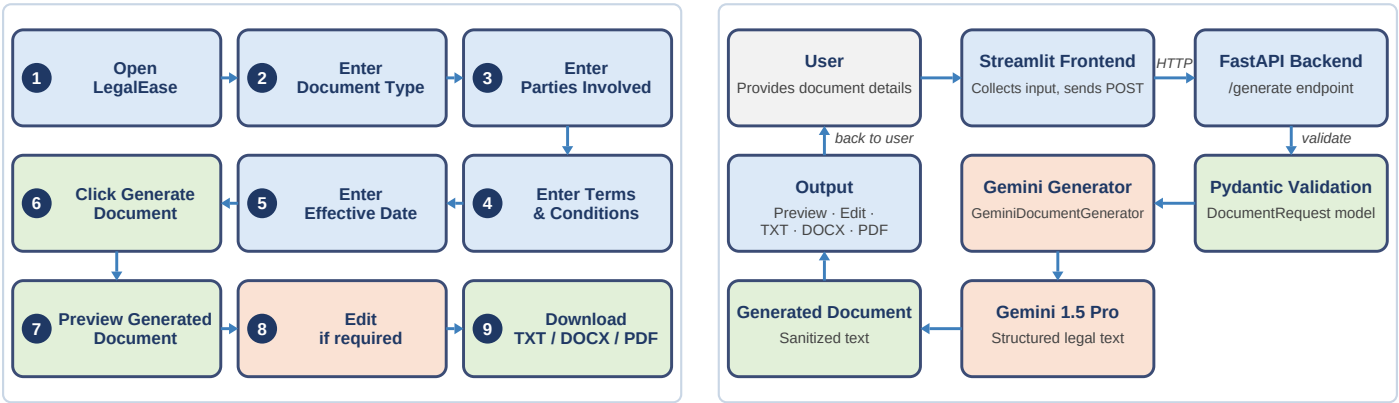
User Flow: Open LegalEase → Enter Document Type → Enter Parties Involved → Enter Terms & Conditions → Enter Effective Date → Click Generate Document → Preview → Edit if required → Download TXT/DOCX/PDF.

Four Main Inputs

- Document Type — specifies the type of legal document.
- Parties — contains the parties involved.
- Terms — contains terms and conditions.
- Dates — provides the effective date.

Folder Structure

- ai_core/ — Gemini-based document generation logic.
- frontend/ — Streamlit interface and formatting utilities.
- legalEaseAPI/ — FastAPI application and API routes.
- Image/ — optional application logo.
- requirements.txt — required Python packages.
- .env.example — environment-variable template.
- README.md — project information and instructions.



AI Legal Document Generator

Document Type (Ex: Agreement, Contract, NDA)

Freelance Work Contract

Parties Involved

Jane Doe (Service Provider), TechNova Inc. (Client)

Terms & Conditions (Use semicolons for bullet points)

Work must be delivered by May 15, 2025; Payment within 7 days of invoice; Confidentiality must be maintained at all times

Streamlit input interface

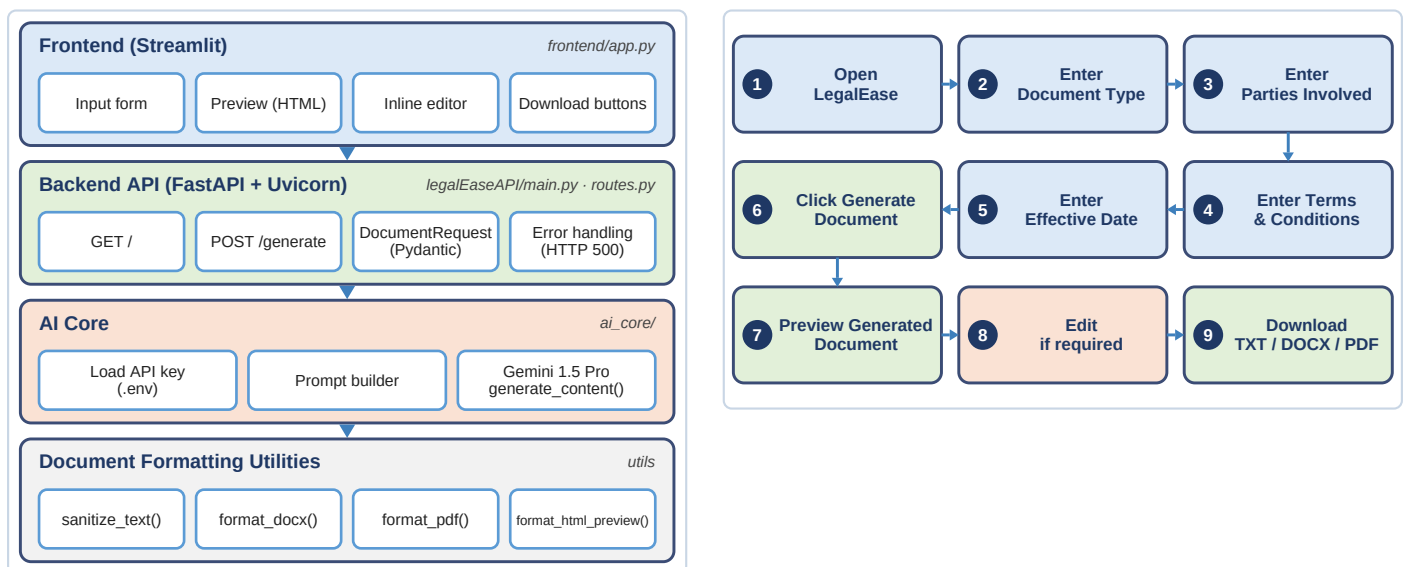
Project Setup & Starting the Application

Setup Steps

- Create virtual environment: `python -m venv venv`
- Activate it on Windows: `venv\Scripts\activate`
- Install dependencies: `pip install -r requirements.txt`
- Rename `.env.example` to `.env` and add `GEMINI_API_KEY`.
- Place a PNG logo at `Image/Logo.png` if required.

Start the Application

- Backend: `uvicorn legalEaseAPI.main:app --reload`
- Frontend: `streamlit run frontend/app.py`
- Documented development address: `http://127.0.0.1:8000`
- Documented local frontend address: `http://localhost:8501`



The screenshot shows the VS Code editor with the project files and the terminal output. The terminal displays the command to run the Streamlit application and the resulting output, including the local and network URLs.

```
LegalEase > legalEaseAPI > _init_.py > ... LegalEase > frontend > utils.py > ...
```

```
1 Generate code (Ctrl+I), or
2 select a language (Ctrl+
3 I) > Start typing to
4 dismiss or don't show this
5 again.
6
7
8 import io
9 import re
10
11 from docx import Document
12 from docx.shared import Inches, Pt
13 from docx.enum.text import WD_ALIGN_PARAGRAPH
14 from fpdf import FPDF
15
16 from config import WEB_LOGO_PATH
17
18
19 def sanitize_text(text: str) -> str:
```

```
PS C:\Users\acer\Downloads\LegalEase (2)\LegalEase> streamlit run frontend/app.py
2026-09-24 21:23:43.653 Uvicorn server started on ::1:8503

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8503
Network URL: http://10.195.63.34:8503

Help agents write better Streamlit apps?
Install the official Streamlit skills in your terminal.
```

The screenshot shows the PowerShell terminal with the command to activate the virtual environment and the resulting output.

```
Warning: PowerShell detected that you might be using a screen reader and has disabled PSReadLine for compatibility purposes. If you want to re-enable it, run 'Import-Module PSReadLine'.

PS C:\Users\acer\Downloads\LegalEase (2)> "c:\Users\acer\Downloads\LegalEase (2)\venv\Scripts\activate"
c:\Users\acer\Downloads\LegalEase (2)\venv\Scripts\activate
PS C:\Users\acer\Downloads\LegalEase (2)>
```

Virtual-environment activation

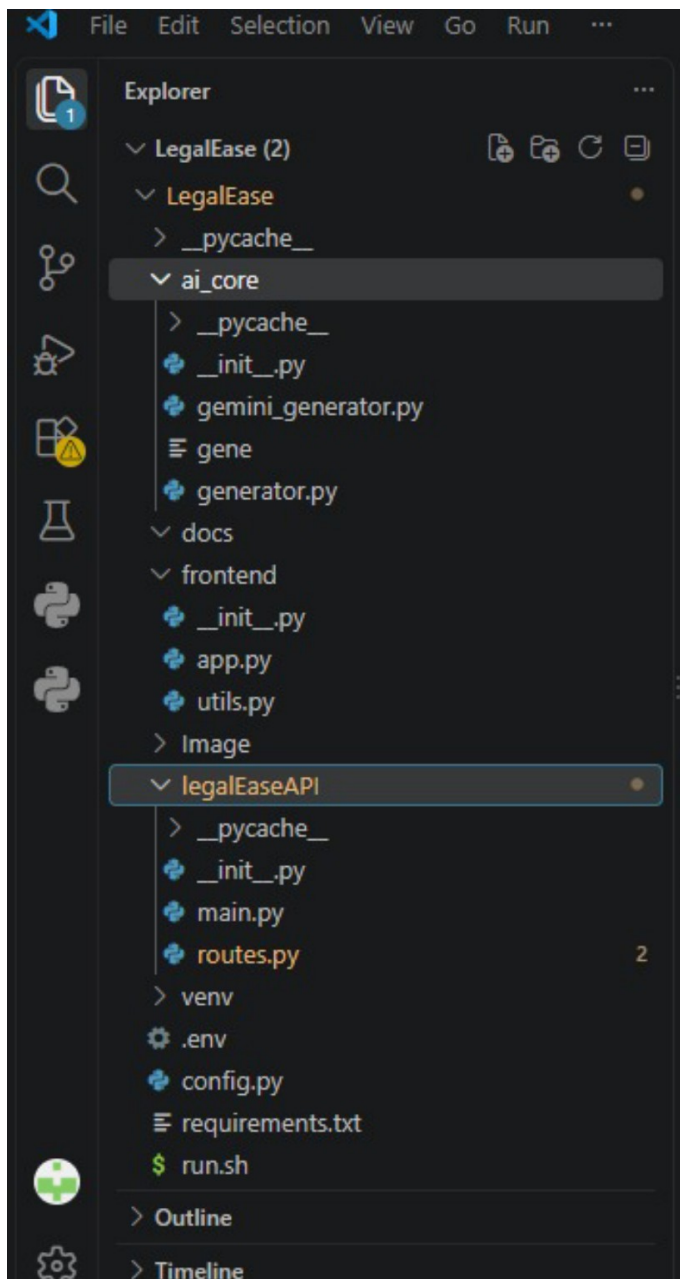
API Endpoints & Backend Development

API Endpoints

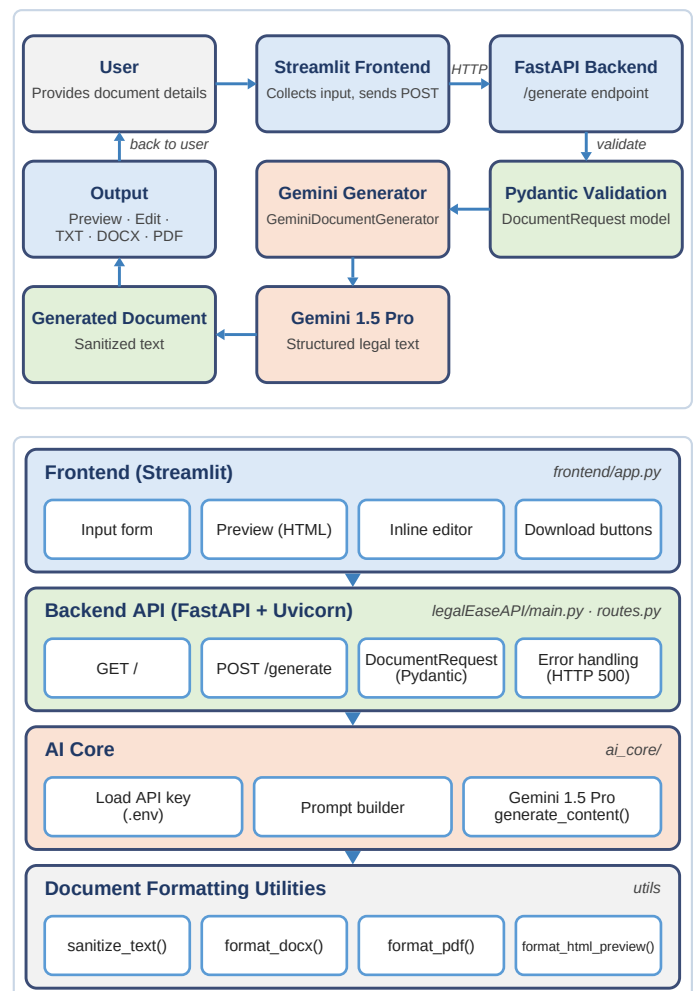
- GET / — API health-check/root endpoint.
- POST /generate — generates a legal document.
- Request fields: document_type, parties, terms, dates.
- The backend returns the generated document in a JSON response using the document field.

Backend Development

- main.py creates the FastAPI application, includes the API router and provides the root health check.
- routes.py defines the DocumentRequest Pydantic model and validates the four input values.
- /generate receives DocumentRequest and calls GeminiDocumentGenerator.generate_document().
- If generation raises an exception, the API returns HTTP 500 with error detail.



FastAPI backend files and project structure



AI, Frontend & Document Formatting

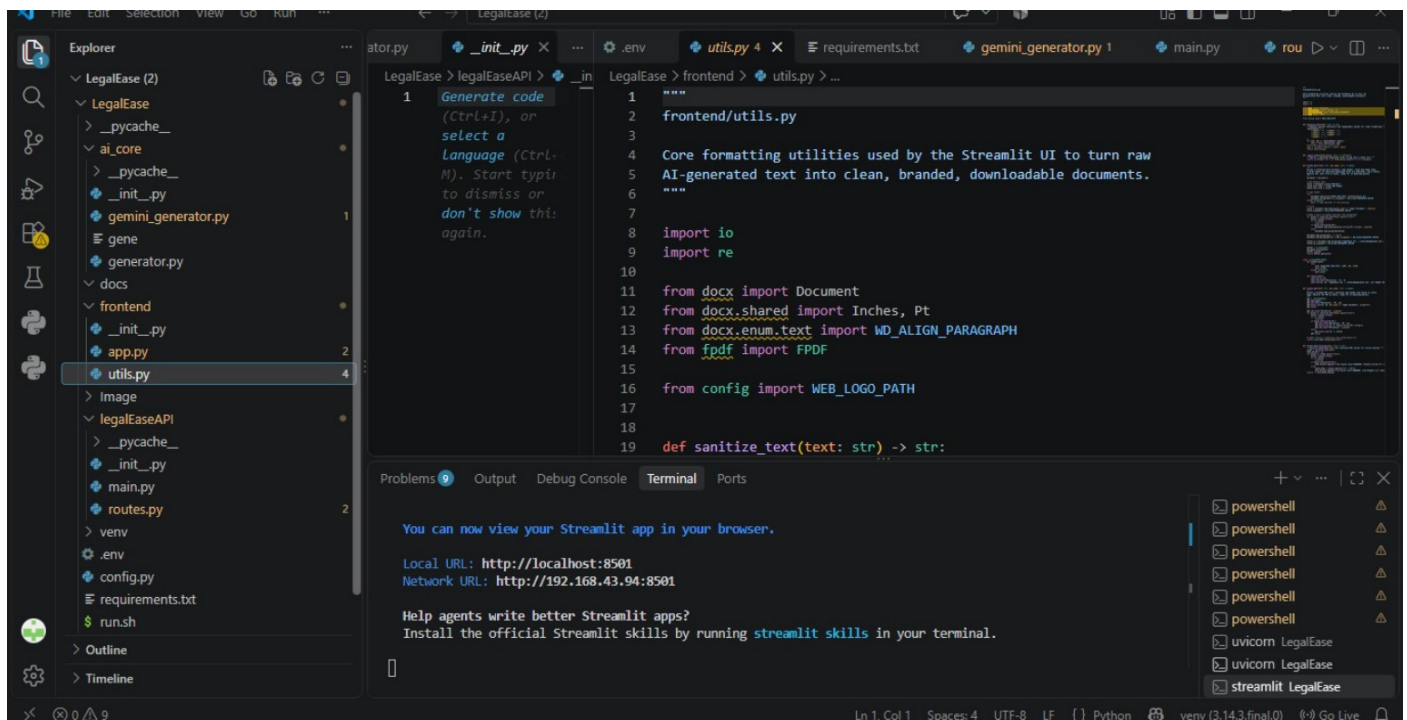
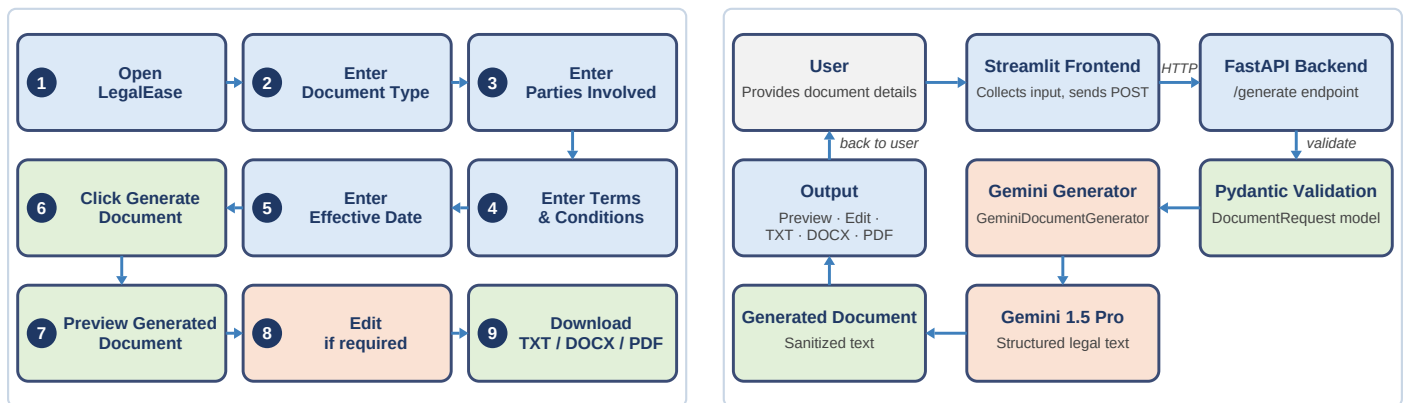
AI Development

- GeminiDocumentGenerator loads environment variables and checks GEMINI_API_KEY.
- It configures the Google Generative AI library and initializes Gemini 1.5 Pro.
- The prompt contains document type, parties, effective date, terms and conditions.
- The generator requests a formal legal structure with sections and clauses.
- The process loads the key, configures the service, builds the prompt, calls generate_content(), and returns the response.

Frontend: app.py creates the LegalEase interface, displays the logo when available, collects inputs, sends the request, stores the generated document in session state, and provides a styled preview/editing area.

Formatting & Export

- Text sanitization replaces incompatible typographic characters.
- DOCX uses a centered title and Times New Roman 11 pt normal text.
- PDF uses fpdf2, centered title, headings and bullet rendering.
- HTML preview converts line breaks for display.
- Download options: TXT, DOCX and PDF.



Document-formatting utility implementation

Execution, Expected Output & Conclusion

Project Execution

- Open the LegalEase project folder in VS Code.
- Create and activate the virtual environment.
- Install dependencies and configure GEMINI_API_KEY.
- Start FastAPI backend and Streamlit frontend.
- Enter document details and generate the legal document.
- Preview/edit the result.
- Download the final document as TXT, DOCX or PDF.

Expected Output: An AI-generated legal document based on the information supplied by the user. The generated content is displayed in the LegalEase interface and can be edited before export.

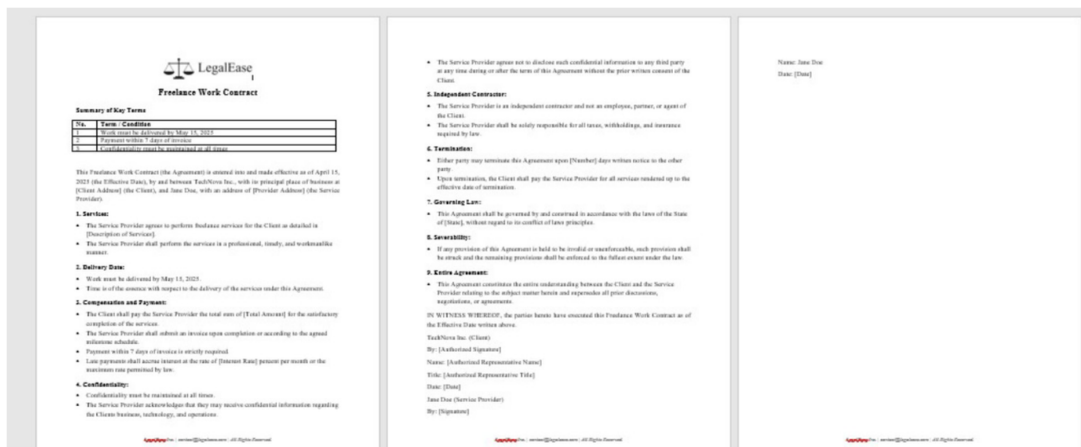
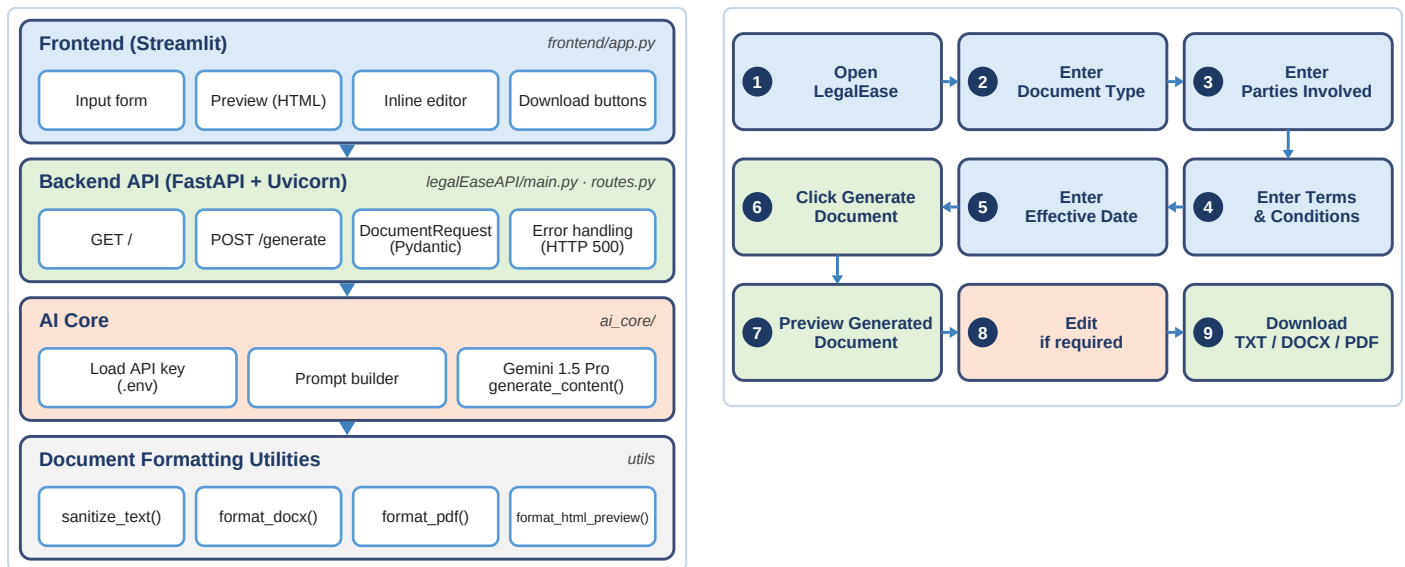
Conclusion

LegalEase combines a Streamlit frontend, FastAPI backend, and Gemini 1.5 Pro to provide an AI-assisted legal document generation workflow. Its modular structure separates the user interface, API layer, AI generation logic, and document-formatting utilities.

Final Workflow: USER INPUT → STREAMLIT → FASTAPI → PYDANTIC VALIDATION → GEMINI 1.5 PRO → GENERATED LEGAL DOCUMENT → PREVIEW / EDIT → TXT / DOCX / PDF

GitHub Link

<https://github.com/<your-username>/LegalEase>



Final generated document with signature section